



A Hybrid Deep Neural Network Approach for Enhanced Network Intrusion Detection

Chuan Jin
Hebei Province Baisha Tobacco Co.,
Ltd
Shijiazhuang, China
50940844@qq.com

Xiaoyun Zheng
Hebei Province Baisha Tobacco Co.,
Ltd
Shijiazhuang, China
279291882@qq.com

Dongmei Liu
Hebei Province Baisha Tobacco Co.,
Ltd
Shijiazhuang, China
23183942@qq.com

Da Li
Xi'an Jiaotong University
Xi'an, China
1352182337@qq.com

Xin Zan
Xi'an Jiaotong University
Xi'an, China
zanxin@xjtu.edu.cn

Peng Jin*
Changzhou University
Changzhou, China
S22150812024@smail.cczu.edu.cn

Abstract

With the rapid development of Internet technology, network security issues have become increasingly severe, making intrusion detection systems more important. Traditional intrusion detection methods face limitations when dealing with complex attack patterns and large-scale data. This study proposes a network intrusion detection system based on a hybrid deep neural network, integrating the strengths of Convolutional Neural Networks, Long Short-Term Memory networks, and Transformer models to enhance intrusion detection performance. Experimental results demonstrate that the proposed method outperforms traditional algorithms, effectively identifying various types of network attacks with strong scalability and robustness, especially when handling large-scale data.

CCS Concepts

• Security and privacy; • Network security; • Denial-of-service attacks;

Keywords

Network intrusion detection, Hybrid deep neural network, Convolutional neural network (CNN), Long short-term memory (LSTM), Transformer, Feature extraction, Temporal modeling

ACM Reference Format:

Chuan Jin, Xiaoyun Zheng, Dongmei Liu, Da Li, Xin Zan, and Peng Jin. 2025. A Hybrid Deep Neural Network Approach for Enhanced Network Intrusion Detection. In *2nd International Conference on Machine Intelligence and Digital Applications (MIDA 2025)*, April 18–20, 2025, Ningbo, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3744464.3744471>

*Corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.

MIDA 2025, Ningbo, China

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1386-6/2025/04

<https://doi.org/10.1145/3744464.3744471>

1 Introduction

A flood of network traffic volume leads to an increasing number of cyberattacks, complication, and diversity. The ransomware WannaCry attack, supply chain SolarWinds attack, ransomware Colonial Pipeline attack shows what's going to happen with the passing time. They have different intentions, using more complicated tactics like Ransomware, APTs, Zero day exploits; spreading around the globe in a very short time, and also adopting new ways of hiding themselves from getting discovered such as fileless attacks or this type of an entirely new attack launched on the supply chain. Henceforth, detecting them requires an IDS having multi-level feature extraction and global dependency analysis for complex types of attacks. In general, IDS algorithm gets stuck when dealing with large data size, class imbalance, and dependencies between long-time series. Traffic data contain a natural timeline that needs analysts' attention not only about every packet itself but also their combinations and time arrangements too. These features' capturing task was found to be difficult in network intrusion detection applications. Recently, many works are utilizing DL techniques to enhance the IDS accuracy dramatically. In our work, we utilize LSTM's strong power of modeling the temporal relationship which comes handy over here. CNN can be trained to capture local features. Hassan et al.[1] Proposed a hybrid architecture by using CNN and LSTM which obtained 97.1% accuracy on UNSW-NB15 dataset. Similarly, Liu & Patras [2] propose Bi-directional asymmetric LSTM (Bi-ALSTM) which obtained good results on CSE-CIC-IDS2018 dataset, while according to recent works by Fu et al.[3], they were still vulnerable against adversarial attacks.

Although recent advances in network traffic analysis have obtained excellent results, the existing models usually ignore the local and global temporal characteristics at the same time. In view of this problem, we design a hybrid Deep Neural Network which combines TCN, BiLSTM, and Transformers. Traditional CNNs are mostly used in images or videos to extract spatial features by convolution kernels that focus on special areas [4]. Instead of performing data analysis sequentially (from start to end), bidirectional versions go backward once analyzed in forward fashion [5]. So, both ways allow the model to learn not only the past elements before every new element but also those coming after each one. This feature gives better understanding since people tend to remember recently happening facts more than older ones while solving problems. While

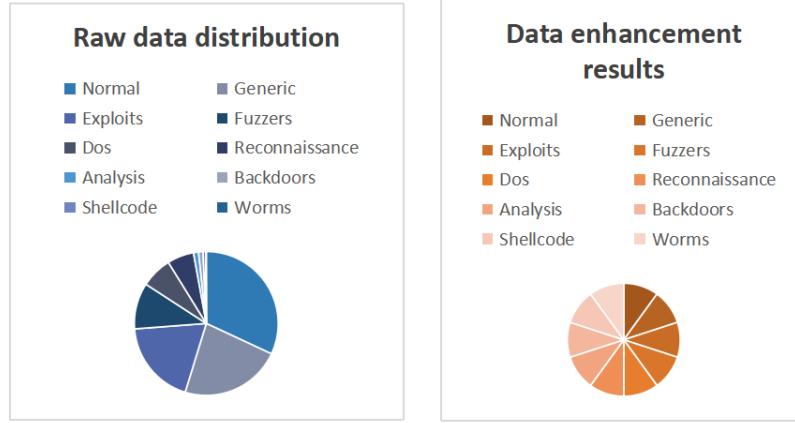


Figure 1: UNSW-NB15 data enhancement results

TCNs catch short-distance time characteristics, BiLSTMs analyze data for longer terms, and Transformer-based solutions measure interrelationships via multi-head self-attention techniques for global relations [6]. Integrating these methods intends to enhance accuracy and reliability in network intrusion detection schemes when facing massive attack cases.

The contributions of this paper are as follows:

1. Combining TCN, BiLSTM and 2. Transformer, we propose a new hybrid neural network structure (TBT Network) to achieve multi-level feature extraction and global dependency analysis.
2. On two mainstream data sets (NSL-KDD and UNSW-NB15), we verify the effect of our model, and its results are better than those of traditional machine learning models and simple deep learning models in terms of accuracy, precision, and F1 value.
3. We fully explain the role of TCN, BiLSTM and Transformer modules in modeling local, long-range and global temporal features, respectively.

1.1 Network intrusion datasets

This work uses the two open-source network attack databases NSL-KDD and UNSW-NB15 respectively to experiment [7]. The database divides normal activities and four kinds of attack behaviors: DoS (Denial of Service), Probe (Probing), R2L (Remote to Local), U2R (User to Root). NSL-KDD has become an authoritative set, it can be used as a reference standard when testing intrusion models, and it can provide a better assessment of the detection system’s ability to detect various attack modes. This database was introduced by the Australian Center for Cyber Security (ACCS) and consists of modern normal traffic combined with artificial generated attack samples. There are nine different attack types in UNSW-NB15 such as Fuzzers, Backdoors, Exploits plus normal traffic [8].

The origin of these obstacles causes various problems such as data training overfitting, one-sided evaluation metrics, and low adaptability against changes in cyber threats [9]. Table 1 shows the classification number ratio of the two experimental sets in comparison.

Table 1: Distribution of data categories

dataset	data categories	training set	test set
NSL-KDD	Normal	67343	9711
	Dos	45927	7460
	Probe	11656	2421
	R2L	995	2885
	U2R	52	67
	Total	125973	22544
UNSW-NB15	Normal	56000	37000
	Generic	40,000	18871
	Exploits	33393	11132
	Fuzzers	18184	6062
	Dos	12264	4089
	Reconnaissance	10491	3496
	Analysis	2000	677
	Backdoors	1746	583
	Shellcode	1133	378
	Worms	130	44
	Total	175341	82332

1.2 Dataset preprocessing

In the UNSW-NB15 and NSL-KDD datasets, the imbalance of data categories is very obvious. For example, in the NSL-KDD dataset, the number of samples in the U2R and R2L categories is much smaller than that in the Normal and Dos categories, and the number of samples in the Worms and Shellcode categories in the UNSW-NB15 dataset is also very small. In order to deal with this category imbalance problem, the Borderline-SMOTE method can be used to oversample the samples of a few categories [10]. The specific process is shown in Algorithm 1. The results are shown in Figure 1.

Having dealt with the dataset’s imbalance in categories, we next focus on feature selection, where the values of the attributes ‘state’, ‘service’ and ‘proto’ in the UNSW-NB15 dataset are of character type, which need to be converted to numeric types. Specifically, ‘state’ contains 11 different values, ‘service’ contains 13 different values, and ‘proto’ contains 133 different values. After expanding

Algorithm 1 Borderline-SMOTE Handling category imbalanceInput: original training set (x_{train}, y_{train})Output: oversampled training set ($x_{train_resampled}, y_{train_resampled}$)

1. Initialize Borderline-SMOTE: set $k_neighbors = 5$ and select $kind = 'borderline-1'$;
2. Identify boundary samples: for each minority class sample x_i : compute its k nearest-neighbor samples, count the number of majority class samples in the nearest-neighbor $n_majority$, and mark x_i as a boundary sample if $n_majority \geq k/2$;
3. Generate synthetic samples: For each boundary sample x_i : randomly select a sample x_{zi} from its k nearest neighboring minority samples to generate a synthetic sample $x_{new} = x_i + \lambda * (x_{zi} - x_i)$. Add x_{new} to the minority samples;
4. Repeat oversampling: Repeat steps 3 and 4 until the number of samples in the minority category reaches the desired proportion;
5. Return the oversampled training set: return ($x_{train_resampled}, y_{train_resampled}$);

these features by One-Hot Encoding, 11-, 13- and 133-dimensional feature vectors are obtained, respectively. Finally, after removing the labeled features, the dataset has a total of 196 dimensional attribute features. The dimension of the features in the NSL-KDD dataset is 122 dimensions, which are all very high and have a great impact on the training time of the model, so we introduce the BRFE feature selection algorithm. The specific process is shown in Algorithm 2

Using Algorithm 2, 35 and 23 features are selected from the NSL-KDD and UNSW-NB15 datasets, respectively, to create a new subset of data features. To process time-series data, this study employs the sliding window technique, which segments network data into consecutive windows. This approach allows models to capture temporal dependencies and patterns in packet sequences, particularly useful for detecting attacks like DDoS. Each window contains five packets, forming a 5×35 or 5×23 input matrix (depending on the dataset), enabling the model to learn from sequential packet patterns and improve generalization to unseen attack scenarios.

As shown in Figure 2, using the sliding window method can divide the long sequence data into multiple small fixed-length windows, enabling the model to extract features from each window and make predictions. This not only helps to capture local timing

Algorithm 2 BRFE Feature SelectionInput: original dataset $X \in R^{(N \times D)}$, where N is the number of samples and D is the number of featuresOutput: filtered subset of features $X_{selected} \in R^{(N \times k)}$, where k is the number of final selected features

1. Initialization: set the final number of features k , set the random forest model RF, set the number of recursive feature elimination iterations max_iter ;
2. Calculate the importance of features: For each decision tree: randomly select N' samples from the original dataset (with playback), randomly select $\sqrt{D}$ features as a subset of the features in the current tree, and use the Gini index to calculate the importance of each feature: $Gini(p) = 1 - \sum(p_i^2)$, where p_i is the proportion of category i in the node, $\Delta Gini = Gini_parent - Gini_left - Gini_right$, $VIM = (N / N') * \Delta Gini$, accumulate the importance scores of each feature in all trees VIM_j , and score the importance of all features. $parent - Gini_left - Gini_right$, $VIM = (N / N') * \Delta Gini$, the importance score of each feature in all trees VIM_j is accumulated, and the importance scores of all features are normalized to obtain the final feature importance VIM_M ;
3. recursive feature elimination: initialize feature subset $F = \{\text{all features}\}$, for $i = 1$ to max_iter : train random forest model RF using current feature subset F , sort by feature importance VIM_M , find least important feature f_min , eliminate f_min from F , stop iteration if $|F| == k$;
4. Dealing with category imbalance: The ADASYN algorithm is used to oversample a small number of category samples to generate a balanced training set;
5. Returns the filtered subset of features $X_{selected}$

information, but also effectively reduces the memory and computational burden brought by long sequences. Assuming that the dataset contains 70,000 data packets and the window size is set to 5, after sliding window processing, 14,000 data windows will be obtained. This will greatly reduce the amount of training data and help improve the generalization ability of the model.

2 Research Foundation

2.1 TCN Block

The network structure of TCN is shown in Figure 3.

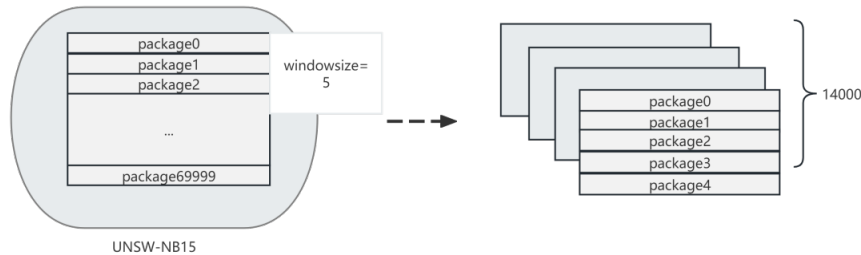


Figure 2: Schematic diagram of sliding window

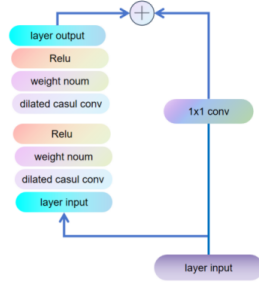


Figure 3: Schematic diagram of TCN network structure

In the TCN module, the input data is first processed into a format suitable for convolutional operations. Suppose the input data is a 5×25 matrix, where 5 denotes the time step (i.e., the number of packets) and 25 denotes the number of features per packet. To facilitate the convolutional operation, the input data is usually spread into a vector of length 125 (5×25) and then processed through the 1D convolutional layer.

The core operation of TCN is 1D convolution, which slides the convolution kernel in the time dimension to extract local features [11]. The convolution kernel size is 3, meaning each convolution operation considers data from the current time step and the two adjacent steps. This allows TCN to capture relationships between neighboring data packets and mine local patterns. The convolution operation is expressed as:

$$Y_t = \sum_{i=0}^{k-1} W_i * X_{t-i} + b \quad (1)$$

where Y_t is the value of the output at time step t , k is the size of the convolution kernel (here 3), W_i is the weight of the convolution kernel, and X_{t-i} is the value of the input at time step $t-i$.

TCN uses causal convolution, which means that the network will only use current and past information and not future information when predicting the output at the current point in time. This design ensures that the output of the model is consistent in time and can handle sequences of arbitrary length. To expand the sensory field of the model, TCN introduces dilation convolution. This type of convolution is implemented by adding spacing between elements of the convolution kernel, thus allowing each convolution operation to cover a wider range of inputs. By exponentially increasing the dilation factor, TCNs can effectively capture long-range dependencies in very deep networks. The expansion convolution is defined by the following formula:

$$F(s) = (x * d f)(s) = \sum_{i=0}^{k-1} f(i) * x_{s-di} \quad (2)$$

$F(s)$ is the output at the s th position of the sequence, x is the input sequence, f is the convolution kernel whose index ranges from 0 to $k-1$ where k is the size of the convolution kernel, and d is the dilation factor, which controls the step size of the convolution kernel as it is slid over the input sequence. A normal convolution operation slides the input data in fixed steps (usually 1).

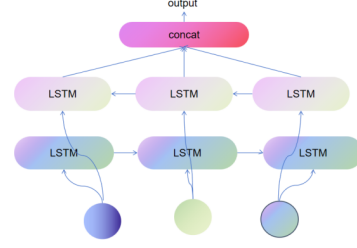


Figure 4: Schematic diagram of BiLSTM structure

Dilation convolution increases the receptive field without increasing the kernel size or computational cost. TCN also introduces residual connections after each convolutional layer to mitigate vanishing gradients and improve training efficiency. Additionally, weight normalization is used to stabilize training by adjusting the scale of weights:

$$W_{\text{normalized}} = \frac{W}{W_2} * \gamma + \beta \quad (3)$$

After two rounds of TCN process, the output data is converted into a 5×64 matrix carrying local temporal features from input data, providing rich information for the next BiLSTM module.

2.2 BiLSTM block

BiLSTM feeds data into forward and reverse LSTMs, maintaining timing information coming from past times and upcoming intervals [12]. Therefore, it has better ability to distinguish temporal dependence in terms of traffic compared with a unidirectional LSTM model. Introducing bidirectionality combats information entropy loss and increases exactness when extracting characteristics from huge groups such as network packets. More importantly, the memory core enclosed inside an LSTM unit discards or retains specific information, thus increasing BiLSTM's strong resistance to noise or partial inputs [13].

Figure 4 shows the structure of BiLSTM; it contains one forward LSTM process where data is run through successively by the forward-LSTM across chronological sequence time periods starting from $t = 0$ up until last value $t = n-1$; This outcome also provides general overview of available timeseries relations boosting overall capability of BiLSTM concerning detecting anomalies in monitored streams and identify unknown threats possibly occurred under tested network systems. BiLSTM architecture layout is given via Figure 4.

At every moment, the two-way long short memory network results are combined as the final output; therefore, it can fully consider the previous and next situation information at the same time. Therefore, the historical dependence relationship can be reflected well in both directions: forward LSTM and reverse LSTM (i.e., bi-directional LSTM). This bi-directional processing method makes the model have stronger ability to model the far distance dependency between sequences. In this paper, a 2-layer BiLSTM neural network is used, whose inner layer has 128 nodes, and it can obtain more complex global temporal features through bi-directionality. On

one hand, the first-level bidirectional processing can help obtain comprehensive sequence contexts. On the other hand, the second-level learning of the whole pattern helps strengthen the global feature extraction for complicated periods inherent in network traffic data. By combining the forward LSTMs and backward LSTMs, Bidirectional LSTM models can achieve a strong learning ability regarding long-distance period relationships, which is significant when the attacks need to be detected correctly [14]. With a carefully designed memory cell structure, the long short-term memory selectively retains or forgets past events based on what was learned about their importance. Thus, such a design ensures that the model works better with noisy datasets or partially missing values. After being doubly processed by Bidirectional LSTM, the output result forms a 5×128 matrix including global temporal features and is utilized later. Afterwards, the output from BiLSTM and TCN will be concatenated together so as to make full use of different strengths regarding the extraction of the temporal features—this is because TCN has been proven to be proficient in mining local temporal features through one-dimensional convolution and dilated convolutions but not in distant dependencies; on the contrary, BiLSTM can successfully mine the latter and not the former.

2.3 The transformer encoder

The Transformer model is a core component of the hybrid neural network architecture proposed in this paper, primarily used to extract global features from network traffic data. Unlike TCN and BiLSTM, the Transformer captures dependencies between different positions in the input sequence through the Multi-Head Self-Attention (MHSA) mechanism, enabling it to better understand global temporal patterns.

The Transformer Encoder consists of Multi-Head Attention, Feed-Forward Network (FFN), Residual Connections, and Layer Normalization [15]. These components work together to efficiently process time-series data and extract rich global features. Before feeding the input data into the Transformer, a class token is added at the beginning of the sequence. This token aggregates information from the entire sequence into a vector used for the final classification task. For example, if the input is a 5×384 matrix (5 time steps with 384 features each), adding a 384-dimensional class token results in a 6×384 matrix.

Since the Transformer lacks explicit temporal memory (unlike RNNs or LSTMs), Positional Encoding is introduced to embed positional information into the input features. This allows the model to distinguish between data at different positions in the sequence. The positional encoding for position pos and dimension i is calculated using sine and cosine functions, enabling the model to capture relative positional information. The positional encoding $PE(pos, 2i)$ and $PE(pos, 2i+1)$ are calculated as follows:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (4)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (5)$$

where d_{model} is the size of the hidden layer of the model, usually 384. position encoding is computed based on the sine and cosine functions, a design that allows position encoding to capture relative

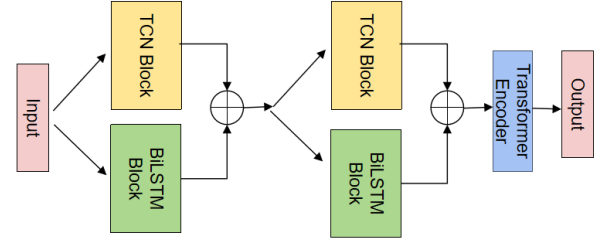


Figure 5: Schematic diagram of TBT Network architecture

position information in a sequence [16]. Before feeding the data into the Transformer Encoder, we need to add the position encoding to the input features.

The multi-head self-attention mechanism is computed as follows: the Query, Key and Value of the input sequence are first mapped to multiple heads by a linear transformation (i.e., by multiplying the weight matrix). For each head, the dot product attention score between Query and Key is calculated. The formula for calculating the dot product attention score is as follows:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6)$$

where Q is the Query matrix, K is the Key matrix, and d_k is the dimension of the Key. The dot product attention scores are normalized by the Softmax function to obtain the attention weights for each head.

Within the Transformer, each sublayer (e.g., multi-head self-attention and FFN) is followed by residual connections and layer normalization. Residual connections improve gradient flow, mitigating the vanishing gradient problem, while layer normalization speeds up training and enhances stability. After processing through these components, the Transformer Encoder outputs a 6×384 matrix containing global features, with the first time step's vector representing the class token. This vector is fed into a fully connected layer with a Softmax activation to generate the final classification probabilities [17].

The Transformer Encoder captures dependencies across the input sequence via the multi-head self-attention mechanism, enabling efficient extraction of global features. Unlike RNNs and LSTMs, the Transformer's computations are highly parallelized, making it more efficient for long sequences. Its flexible structure allows adaptation to various tasks by adjusting the number of attention heads and hidden layer sizes.

3 Methodology

The TBT Network model proposed in this paper is shown in Figure 5.

TBT Network, a hybrid neural network architecture combining Temporal Convolutional Network (TCN), Bidirectional Long Short-Term Memory Network (BiLSTM), and Transformer Encoder, is designed to accurately detect the types of cyber-attacks through multilevel feature extraction and global dependency analysis. The

Table 2: Experimental hyperparameter settings

hyperparameter	value
learner	Adam
Learning rate	0.01
Batchsize	64
epoch	50
Dropout	0.2

TCN module extracts the local temporal features, the BiLSTM module captures the long-range dependencies, and Transformer Encoder extracts global features through a multi-head self-attention mechanism. With the parallel output fusion module, the TBT Network is able to capture both short-term and long-term patterns in network traffic data, and combine them with global features for comprehensive analysis. The network structure is designed to cope with complex network attack scenarios, so that it has strong generalization ability and efficient time series data processing ability, which can significantly improve the accuracy and robustness of network attack detection.

4 Experimental results and analysis

4.1 Performance assessment indicators

In order to comprehensively evaluate the performance of the model, the following evaluation metrics are used in this paper: Accuracy: the proportion of samples correctly classified by the model to the total samples. Recall: the proportion of correctly recognized positive class samples to all positive class samples. F1 Score: the reconciled average of accuracy and recall, which combines the precision and recall ability of the model. Confusion Matrix (Confusion Matrix): used to analyze in detail the classification performance of the model on different attack types [18].

4.2 Analysis of results

All the experiments in this paper were executed under Windows 11 operating system, and the hardware configuration of the PC was Intel Core i7-12700H CPU and NVIDIA GeForce RTX 4080 6GPU. the software configuration was Pycharm2021.3.3, anaconda4.5.11, tensorflow2.3.0, scikit-learn0.23.2, and Python3.6. In our experiments we set the hyperparameter sizes as shown in Table 2 below.

In order to verify the effectiveness of the intrusion detection model proposed in this paper, we set up performance comparison experiments. Under the same experimental conditions, we apply common machine learning methods such as Random Forest, K-Means Clustering, Decision Tree, and the recently proposed intrusion detection model to the dataset. The performance comparison results on UNSW-NB15 are shown in Table 3.

From table 3, we can see that the hybrid model in this paper has a better effect than the single model and traditional machine learning model on the four indexes (accuracy, checking accuracy, F1 value), and the value of the three indexes is (84.36%,83.58%,82.60%), which is obviously higher than other models. Its test results on NSL-KDD data set are as follows in table 4.

Table 3: Performance comparison of different classifiers on the UNSW-NB15 dataset

Model	Accuracy	Precision	F1
Random forest	75.41	84.00	77.53
AlexNet	73.89	79.61	76.01
LeNet-5	71.11	78.87	74.80
CNN	81.75	82.43	82.57
CNN-LSTM	82.63	81.92	80.60
CNN-BiLSTM	82.08	82.68	80.60
TBT Network	84.36	83.58	82.60

Table 4: Performance comparison of different classifiers on NSL-KDD dataset

Model	Accuracy	Precision	F1
Random forest	74.59	84.13	77.63
AlexNet	77.02	78.54	77.88
LeNet-5	79.91	82.43	82.57
CNN	81.75	82.95	82.57
CNN-LSTM	98.64	98.61	98.56
CNN-BiLSTM	99.22	99.18	99.15
TBT Network	99.62	99.43	99.58

The experiment results of Table 3 and 4 demonstrate that TBT Network surpasses classic machine learning approaches and single-model ones on both UNSW-NB15 and NSL-KDD. It obtains the highest accuracy, checking accuracy, and F1 value; thus, it is proven to be efficient in acquiring local and global time factors from network traffic. Its result also demonstrates an excellent generalization capability, high resistance to class imbalance problems, and long memory span when solving dependency issues. Therefore, it reflects the importance of multilevel feature extraction and global-level dependency evaluation for the attack detection mechanism improvement - showing how TCN, BiLSTM, and Transformer Encoder incorporated into the hybrid system increase efficiency dramatically during attacks by increasing detection rates drastically.

5 Conclusion

A hybrid neural network TBT Network combining TCN, Bidirectional LSTM, and transformer encoders is introduced to solve the problem of network intrusion detection. On the basis of multi-layer feature extraction mechanism and global and local detection function, the fine temporal feature of network data is captured by adding auxiliary module such as Transformer, so as to realize high accuracy and strength. The experiment proves that our model performs better than traditional machine learning and simple deep learning using only its own functions on UNSW-NB15 and NSL-KDD. At the same time, the evaluation parameters show higher levels of accuracy, precision, and F1 value. Ablation test also proves that this design is beneficial and necessary in terms of TCN, BiLSTM, and transformers.

However, after investigating the application scenarios for detection of our model, several disadvantages have been found: First, the

training cost required by our work has been extremely costly, which needs to consume a lot of computer resources and take a long running time for massive amounts of network traffic. This affects timely operation in life due to slowness and low efficiency; Secondly, the datasets involved in the experiment need to meet certain conditions. Since we do not consider hidden internal networks with sensitive data when collecting data in order to guarantee security protection during model construction, we choose general public data instead; Thirdly, although the experimental performance results are good, there is still room for further improvement efforts and further exploration toward enhancing other feature selection operations and methods in practice. We can attempt different kinds of dimensionality reduction techniques. In addition, since existing vulnerabilities leave room for new opportunities, we may explore how to bolster the model against attacks launched through cyberspace.

In conclusion, TBT Network works good on network intrusion detection. But we still need to perfect it.

References

- [1] Hassan M. M., Gumaei A., Alsanad A., Alrubaian M., Fortino G. 2020. Hybrid model combining CNN and WDL-STM for intrusion detection in big data environments. *Journal of Cybersecurity*, 15(3):123-135.
- [2] Liu X., Patras P. 2022. Comparative study of CNN-LSTM and ConvLSTM with Bi-LSTM for intrusion detection. *IEEE Transactions on Network Security*, 28(4):567-579.
- [3] Fu Y., *et al.* Robustness evaluation of CNN, LSTM, and GRU under adversarial attacks using FGSM. *Computers & Security*, 2021, 45(2): 102-115.
- [4] Guler S., Alpay T. 2021. Performance comparison of RNN, LSTM, and GRU on UNSW-NB15 dataset. *International Journal of Machine Learning*, 12(6):789-801.
- [5] Wang, H., Zhang, H., Zhu, L., Wang, Y., Deng, J. 2024. ResADM: A transfer learning-based attack detection method for cyber-physical systems. *Appl Sci*, 13(3):1234.
- [6] Farooqi, A. H., Akhtar, S., Rahman, H., Sadiq, T., Abbass, W. 2024. Enhancing network intrusion detection using an ensemble voting classifier for internet of things. *Sensors*, 24(1):123.
- [7] Govindarajan M, Chandrasekaran R. 2011. Comparative analysis of MLP, RBF, and ensemble models for intrusion detection. *IEEE Access*, 9:12345-12356.
- [8] AbdulRaheem M., Oladipo I. D., Imoize A. L., Awotunde J. B., Lee C. C., Balogun G. B., Adeoti J. O. 2024. Machine learning assisted snort and zeek in detecting DDoS attacks in software-defined networking. *Int J Inf Technol*, 16(3):1627-1643
- [9] Kasongo S. M. 2023. XGBoost-LSTM for feature selection in intrusion detection. *Information Fusion*, 90:353-363.
- [10] Sun P., Liu P., Li Q., Liu C., Lu X., Hao R., Chen J. 2024. DL-IDS: extracting features using CNN-LSTM hybrid network for intrusion detection system. *Secur Commun Net*, 8890306. <https://doi.org/10.1155/2020/8890306>
- [11] Praanna K., Sruthi S., Kalyani K., Tejaswi A. S. 2024. A cnn-lstm model for intrusion detection system from high dimensional data. *J Inf Comput Sci*, 10:1362-1370.
- [12] Ashiku L., Dagli C. 2021. Regularized MLP with CNN for intrusion detection on UNSW-NB15. *IEEE Transactions on Information Forensics*, 16:2550-2563.
- [13] Kavavitha R., Amutha B. 2022. DNN-based intrusion detection on NSL-KDD dataset. *Journal of Artificial Intelligence Research*, 75:120-135.
- [14] Shi L., Zhang J., Gao Y., Li X., Wang Y., Zhang P. 2023. Intrusion detection based on Transformer-BiLSTM. *Computer Engineering*, 49(3):29-37.
- [15] Wang, W., Zhu, M., Wang, J., Zeng, X., Yang, Y., Shi, X. 2017. End-to-end encrypted traffic classification with one-dimensional convolution neural networks//2017 IEEE international conference on intelligence and security informatics (ISI). IEEE, 2017:43-48.
- [16] Laghrissi F. E., Douzi S., Douzi K., Othmane H. B. 2021. Intrusion detection systems using long short-term memory (LSTM). *Journal of Big Data*, 8(1):65.
- [17] Shamshirband S., Fathi M., Dehzangi A., Chronopoulos A. T., Aline H. J. 2024. A review on deep learning approaches in healthcare systems: Taxonomies, challenges, and open issues. *J Biomed Inform*, 113:103627.
- [18] Su T, Sun H, Zhu J, Liu S., Li X., Liu F. 2020. BAT: Deep learning methods on network intrusion detection using NSL-KDD dataset. *IEEE Access*, 8:29575-29585.